User Requirements Specification (URS)

Media Library

Vera Utina



Date: 13-02-25
Version: 1.0
State: Start Version
Author : Utina, Vera V.A.

Version history

Version	Date	Author(s)	Changes	State
1.0	13-02-25	Utina, Vera V.A.	Initial beginning on the URS	Not Finished

Intro

This document defines what the platform needs to do (Functional requirements), how it should perform (Non-functional requirements), and how users will interact with it (Use cases).

Functional Requirements (FR)

	Priority
FR1. Artists can create new media by entering details. **Q01-01.** The required fields are title, description, genre, image file and audio file. **C01-01.** Artists must provide at least one genre for each media item.	**Must**
FR2. Artists can view their own media list, with search and filter options **C02-01**. Artists can only filter their own media.	**Must**
FR3. Artists can update existing media details. **C03-01**. Artists can only edit their own media items.	**Must**
FR4. Artists can delete their own media from the platform when it is no longer relevant. **C04-01.** Artists can only delete their own media items.	**Must**
FR5. Artists can update their personal information, such as password, email, and name. **C05-01.** Artists must provide their current password to update sensitive information (e.g., email, password). **C05-02.** Email updates must not allow duplicate emails already registered in the system.	**Must**
FR6. Artists can monitor their popularity. **C06-01:** Popularity metrics should be based on predefined factors such as play count and likes. **C06-02:** Popularity tracking should provide real-time or periodic updates.	**Could**
FR7. Artists can monitor user activity on their published media. **C07-01:** Activity tracking must include views, likes and reviews. **C07-02:** Artists must not have access to personal listener information.	**Could**
FR8. Artists can filter their own media. **C08-01:** Filters must include genre and upload date. **C08-02:** Filtering must be limited to the artist's own media content.	**Should**
FR9. Listeners can browse artists' media.	**Should**
FR10. Listeners can view artists' media.	**Must**

C10-01: Popularity count should be updated only once when the media is played.

FR11. Listeners can review artists' media. **Could**

FR12. Listeners can provide feedback on artists' media. **Could**
C12-01. Neither Artist nor Listener cannot modify the feedback.

FR13. Listeners can search for media by title or artist. **Should**

FR14. Listeners can filter media by genre. **Should**
C14-01. Filters must support multiple genres and allow combination filtering.

FR15. Listeners can create their own playlists by providing details such as title and a list of media. **Must**
Q15-01. A playlist requires a title, description, privacy option and at least one media item.
C15-01. Listeners are not able to add artists' collections to their playlists.

FR16. Listeners can edit their playlists, including updating the title and modifying the list of media. **Must**
C16-01. Listeners can only modify their own playlists.

FR17. Listeners can delete their playlists if necessary. **Must**

FR18. Listeners can add specific media to their favorite playlist. **Should**

FR19. Listeners can remove specific media from their favorite playlist. **Should**

FR20. Users can delete their accounts if necessary. **Could**
C20-01. Associated data, such as playlists and reviews, should also be removed.

FR21. The system allows users to sign up as either an Artist or a Listener. **Must**
C21-01. Users must select their role (Artist or Listener) during sign-up.
C22-02. Secure login mechanisms should be provided for both customers and administrators, requiring **strong passwords (minimum 14 characters, including one upper and low characters)**.
C22-03. All sensitive data (passwords) must be secured **using BCript.Net**.

FR22. The system can authenticate users. **Must**

FR23. The system recommends playlists based on the listener's preferences. **Could**

FR24. The system analyzes an artist's popularity based on their published media content. **Could**

FR25. Artists can organize their media into collections. **Must**
C25-01. Artists can categorize their media into collections such as Albums and Audiobooks.
C25-02. The system must verify the media type before adding it to a collection.
C25-03. A collection requires a title and at least one media item.

FR26. Artists can update their collections. **Must**

FR27. Artists can delete their collections. **Must**
C27-01. A media item in a collection cannot be deleted along with the collection.

Non-Functional Requirements (NFR)

NFR1. The system should respond to user actions, including page loads **within 3 seconds**..
NFR2. The system should log all actions performed by users to ensure accountability and security **for a minimum of 5 years**.
NFR3. The system should have an intuitive interface, making navigation easy for users, **requiring a maximum of 4 clicks for primary tasks**.
NFR4. The website should be responsive, providing a consistent user experience on all devices **(desktops, tablets, and smartphones)**.
NFR5. Detailed documentation should be provided to guide developers in maintaining and updating the system.

Use Cases

Use Case 1: Registering

Use Case Element	Details
Use Case Name	Registering
Functional requirements	FR21;
Actor	User
Preconditions	
Main Flow (Success Scenario)	1. The System displays the Main Page. 2. The User navigates to the login page. 3. The User selects the registration option. 4. The System displays the registration form. 5. The User enters the required information[1][2]. 6. The System confirms the registration.
Extensions	- **1.** Required fields are missing, the system asks for the missing details. Return to step 5. - **2.** The password is not strong enough, the system asks for the new one. Return to step 5.
Postconditions	The User is successfully registered as Artist and added to the System.

Use Case 2: Authentication.

Use Case Element	Details
Use Case Name	Authentication
Functional requirements	FR22;
Actor	Artist
Preconditions	UC1;
Main Flow (Success Scenario)	1. The System displays the Main Page. 2. The Artist navigates to the login page. 3. The Artist enters their login credentials[1][2]. 4. The System confirms the authentication. 5. The Artist is redirected to the dashboard.
Extensions	- **1.** Required fields are missing, the system asks for the missing details. Return to step 3. - **2.** The password/email is not correct, the system asks for the correct information. Return to step 3.
Postconditions	The Artist is successfully authenticated.

Use Case 3: Media Creation.

Use Case Element	Details
Use Case Name	Media Creation
Functional requirements	FR1;
Actor	Artist
Preconditions	UC2;

Main Flow (Success Scenario)	1. The System displays the Artist Page. 2. The Artist navigates to the media creation section. 3. The System shows the Media Form. 4. The Artist enters required information and uploads the files.[1][2] 5. The System confirms the action.
Extensions	- **1.** Required fields are missing, the system asks for the missing details. Return to step 4. - **2.** If the uploaded file format is unsupported, the system requests a valid file. Return to step 4 .
Postconditions	The Media is successfully created.

Use Case 4: Media Modification.

Use Case Element	Details
Use Case Name	Media Modification
Functional requirements	FR3;
Actor	Artist
Preconditions	UC2; Media must exist
Main Flow (Success Scenario)	1. The system displays the artist dashboard. 2. The artist selects a media item for editing. 3. The system presents the media details. 4. The artist updates the necessary information.[1][2] 5. The system verifies and saves the modifications.
Extensions	- **1.** Required fields are missing, the system asks for the missing details. Return to step 4. - **2.** If the updated media file is unsupported, the system requests a valid file. Return to step 4.
Postconditions	The Media is successfully updated.

Use Case 5: Media Viewing.

Use Case Element	Details
Use Case Name	Media Overview
Functional requirements	FR2;
Actor	Artist
Preconditions	UC2;
Main Flow (Success Scenario)	1. The system displays the artist dashboard. 2. The artist selects a media item to view. 3. The system presents the media details. 4. The system streams the media file for playback.
Extensions	
Postconditions	The Media Overview action is successfully performed.

Use Case 6: Media Deletion.

Use Case Element	Details
Use Case Name	Media Deletion
Functional requirements	FR1;
Actor	Artist
Preconditions	UC2;

Main Flow (Success Scenario)	1. The system displays the artist dashboard. 2. The artist selects a media item for deletion. 3. The system asks for confirmation. 4. The artist confirms deletion.[1] 5. The system removes the media from the catalog.
Extensions	- **1.** If the media is part of a collection, the system notifies the artist and requests action.
Postconditions	The Media is successfully deleted.

Use Case 7: Collection Creation.

Use Case Element	Details
Use Case Name	Collection Creation
Functional requirements	FR25;
Actor	Artist
Preconditions	UC2;
Main Flow (Success Scenario)	1. The system presents the artist dashboard. 2. The artist navigates to the collection creation page. 3. The system displays a form to create a collection. 4. The artist provides the required details, including title and media items.[1][2][3] 5. The system verifies and confirms the creation.
Extensions	**1.** If required fields are missing, the system prompts for completion. Return to step 4. **2.** If the media type is incompatible with the collection, the system displays an error. Return to step 4. 3. If the collection title is already in use, the system requests a different name. Return to step 4.

Postconditions	The Collection is successfully created.

Use Case 8: Collection Modification.

Use Case Element	Details
Use Case Name	Collection Modification
Functional requirements	FR26;
Actor	Artist
Preconditions	UC2; Selected collection must exist;
Main Flow (Success Scenario)	1. The system presents the artist dashboard. 2. The artist selects an existing collection to modify. 3. The system displays the collection details. 4. The artist updates the necessary fields.[1][2][3] 5. The system verifies and saves the modifications.
Extensions	**1.** If required fields are missing, the system prompts for completion. Return to step 4. **2.** If an invalid media type is added, the system displays an error. Return to step 4. **3.** If the updated title is already taken, the system requests a different name. Return to step 4.
Postconditions	The Collection is successfully modified.

Use Case 9: Collection Overview.

Use Case Element	Details
Use Case Name	Collection Overview

Functional requirements	FR8;
Actor	Artist
Preconditions	UC2; Selected collection must exist
Main Flow (Success Scenario)	1. The system presents the artist dashboard. 2. The artist selects a collection to view.[1] 3. The system displays the collection details, including media items
Extensions	**1.** If the collection does not exist, the system notifies the artist. Return to step 2.
Postconditions	The Media Overview action is successfully performed.

Use Case 10: Collection Deletion.

Use Case Element	Details
Use Case Name	Collection Deletion
Functional requirements	FR27;
Actor	Artist
Preconditions	UC2; Selected collection must exist.
Main Flow (Success Scenario)	1. The system presents the artist dashboard. 2. The artist selects a collection to delete. 3. The system requests confirmation. 4. The artist confirms the deletion. 5. The system removes the collection from the catalog.[1][2]

Extensions	1. If the collection contains media, the system notifies the artist that the media will not be deleted. Return to step 5. 2. If the collection does not exist, the system displays an error.
Postconditions	The Collection is successfully deleted.

Use Case 11: Media Browsing/Filtering.

Use Case Element	Details
Use Case Name	Media Browsing/Filtering
Functional requirements	FR13, FR14;
Actor	Artist
Preconditions	UC2; The media list must exist.
Main Flow (Success Scenario)	1. The system displays the media library. 2. The listener applies filters or searches by title/artist.[1] 3. The system displays the relevant media.
Extensions	**1.** If no media match the filters, the system displays a message.
Postconditions	The Media Browsing/Filtering action is successfully performed.

Use Case 12: Collection Browsing/Filtering.

Use Case Element	Details

Use Case Name	Collection Browsing/Filtering
Functional requirements	FR9;
Actor	Artist
Preconditions	UC2; The collection list must exist
Main Flow (Success Scenario)	1. The system displays the collection library. 2. The listener applies filters or searches by title/artist.[1] 3. The system displays the matching collections.
Extensions	**1.** If no collections match the filters, the system displays a message.
Postconditions	The Collection Browsing/Filtering action is successfully performed.

Use Case 13: Playlist Creation.

Use Case Element	Details
Use Case Name	Playlist Creation
Functional requirements	FR15;
Actor	Listener
Preconditions	UC2;
Main Flow (Success Scenario)	1. The system displays the listener dashboard. 2. The listener navigates to the playlist creation page. 3. The system presents a playlist creation form. 4. The listener enters the required details.[1] 5. The system verifies and confirms the creation

Extensions	**1.** If required fields are missing, the system prompts for completion. Return to step 4.
Postconditions	The Playlist is successfully created.

Use Case 14: Playlist Modification.

Use Case Element	Details
Use Case Name	Media Modification
Functional requirements	FR16;
Actor	Listener
Preconditions	UC2;
Main Flow (Success Scenario)	1. The system displays the listener dashboard. 2. The listener selects a playlist to modify. 3. The system displays the playlist details. 4. The listener updates the required fields.[1] 5. The system verifies and saves the modifications.
Extensions	**1.** If required fields are missing, the system prompts for completion. Return to step 4.
Postconditions	The Playlist is successfully modified.

Use Case 15: Playlist Overview.

Use Case Element	Details

Use Case Name	Playlist Overview
Functional requirements	FR15;
Actor	Listener
Preconditions	UC2; Playlist must exist
Main Flow (Success Scenario)	1. The system displays the listener's dashboard. 2. The listener selects a playlist to view. 3. The system displays the playlist details, including media items
Extensions	
Postconditions	The Playlist Overview action is successfully performed.

Use Case 16: Playlist Deletion.

Use Case Element	Details
Use Case Name	Playlist Deletion
Functional requirements	FR17;
Actor	Listener
Preconditions	UC2; Selected Playlist must exist
Main Flow (Success Scenario)	1. The system displays the listener's dashboard. 2. The listener selects a playlist to delete. 3. The system requests confirmation. 4. The listener confirms the deletion.[1] 5. The system removes the playlist from the catalog

Extensions	
	1. If the playlist contains media, the system warns the listener before deletion. Return to step 4
Postconditions	The Playlist is successfully deleted.